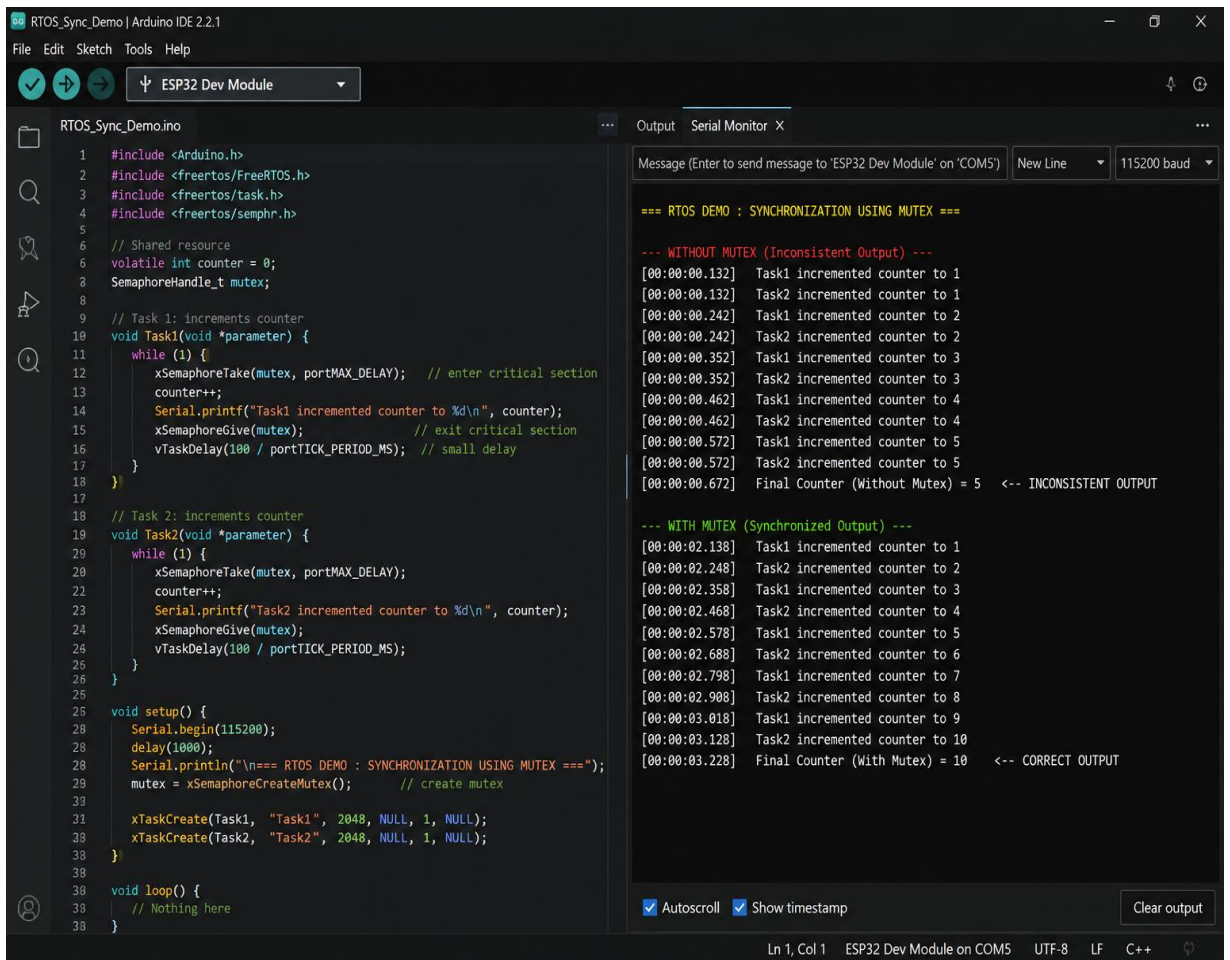


1. Synchronization Using Mutex

Procedure:

- Create two tasks accessing a shared variable.
- Observe the inconsistent output without synchronization.
- Create a **mutex** and lock the shared resource before access.
- Release the mutex after updating the data.
- Verify that the final output is correct.



The screenshot shows the Arduino IDE interface with the file `RTOS_Sync_Demo.ino` open. The code implements two tasks, `Task1` and `Task2`, that increment a shared counter. `Task1` increments the counter 5 times, and `Task2` increments it 5 times. The code uses a semaphore (mutex) to ensure mutual exclusion. The Serial Monitor shows the output of the program, demonstrating the effect of the mutex on the final counter value.

```
1 #include <Arduino.h>
2 #include <freertos/FreeRTOS.h>
3 #include <freertos/task.h>
4 #include <freertos/semphr.h>
5
6 // Shared resource
7 volatile int counter = 0;
8 SemaphoreHandle_t mutex;
9
10 // Task 1: increments counter
11 void Task1(void *parameter) {
12     while (1) {
13         xSemaphoreTake(mutex, portMAX_DELAY); // enter critical section
14         counter++;
15         Serial.printf("Task1 incremented counter to %d\n", counter);
16         xSemaphoreGive(mutex); // exit critical section
17         vTaskDelay(100 / portTICK_PERIOD_MS); // small delay
18     }
19 }
20
21 // Task 2: increments counter
22 void Task2(void *parameter) {
23     while (1) {
24         xSemaphoreTake(mutex, portMAX_DELAY);
25         counter++;
26         Serial.printf("Task2 incremented counter to %d\n", counter);
27         xSemaphoreGive(mutex);
28         vTaskDelay(100 / portTICK_PERIOD_MS);
29     }
30 }
31
32 void setup() {
33     Serial.begin(115200);
34     delay(1000);
35     Serial.println("\n=== RTOS DEMO : SYNCHRONIZATION USING MUTEX ===");
36     mutex = xSemaphoreCreateMutex(); // create mutex
37
38     xTaskCreate(Task1, "Task1", 2048, NULL, 1, NULL);
39     xTaskCreate(Task2, "Task2", 2048, NULL, 1, NULL);
40 }
41
42 void loop() {
43     // Nothing here
44 }
```

Serial Monitor Output:

```
=== RTOS DEMO : SYNCHRONIZATION USING MUTEX ===
--- WITHOUT MUTEX (Inconsistent Output) ---
[00:00:00.132] Task1 incremented counter to 1
[00:00:00.132] Task2 incremented counter to 1
[00:00:00.242] Task1 incremented counter to 2
[00:00:00.242] Task2 incremented counter to 2
[00:00:00.352] Task1 incremented counter to 3
[00:00:00.352] Task2 incremented counter to 3
[00:00:00.462] Task1 incremented counter to 4
[00:00:00.462] Task2 incremented counter to 4
[00:00:00.572] Task1 incremented counter to 5
[00:00:00.572] Task2 incremented counter to 5
[00:00:00.672] Final Counter (Without Mutex) = 5 <-- INCONSISTENT OUTPUT

--- WITH MUTEX (Synchronized Output) ---
[00:00:02.138] Task1 incremented counter to 1
[00:00:02.248] Task2 incremented counter to 2
[00:00:02.358] Task1 incremented counter to 3
[00:00:02.468] Task2 incremented counter to 4
[00:00:02.578] Task1 incremented counter to 5
[00:00:02.688] Task2 incremented counter to 6
[00:00:02.798] Task1 incremented counter to 7
[00:00:02.908] Task2 incremented counter to 8
[00:00:03.018] Task1 incremented counter to 9
[00:00:03.128] Task2 incremented counter to 10
[00:00:03.228] Final Counter (With Mutex) = 10 <-- CORRECT OUTPUT
```

2. Task Communication Using Queue

Procedure:

- Create a **Sensor Task** and a **Logger/Processing Task**.
- Create a message queue between them.
- Send sensor values through the queue.
- Receive and process the values in the second task

The screenshot displays the Arduino IDE interface for the 'RTOS_Comm_Demo' project. The left pane shows the source code, which includes headers for Arduino, FreeRTOS, and queue/task management. It defines a queue of length 10 and two tasks: 'SensorTask' and 'LoggerTask'. The 'SensorTask' generates random data and sends it to the queue every 500ms. The 'LoggerTask' receives data from the queue and prints it, simulating a 300ms processing delay. The 'Setup' function initializes the serial port at 115200 baud and creates the queue and tasks. The 'loop' function is currently empty.

The right pane shows the 'Serial Monitor' output at 115200 baud. It displays a log of messages sent and received between the tasks, demonstrating the queue's FIFO behavior. The output includes a comparison of the queue mechanism with a mailbox, concluding that the queue is the better choice for fast communication with minimal overhead and no data loss.

```
1 #include <Arduino.h>
2 #include <freertos/FreeRTOS.h>
3 #include <freertos/task.h>
4 #include <freertos/queue.h>
5 #include <freertos/semphr.h>
6
7 // Message Queue to send sensor data
8 #define QUEUE_LENGTH 10
9 QueueHandle_t sensorQueue;
10
11 // Task Handles
12 TaskHandle_t SensorTaskHandle;
13 TaskHandle_t LoggerTaskHandle;
14
15 // Sensor Task: generates data and sends to queue
16 void SensorTask(void *pvParameters) {
17     int data = 0;
18     while (1) {
19         data = random(20, 40); // Simulated sensor value
20         if (xQueueSend(sensorQueue, &data, 0) == pdPASS) {
21             Serial.printf("[SensorTask] Sent: %d\n", data);
22         } else {
23             Serial.println("[SensorTask] Queue Full!");
24         }
25         vTaskDelay(pdMS_TO_TICKS(500)); // every 500ms
26     }
27 }
28
29 // Logger Task: receives data from queue and processes it
30 void LoggerTask(void *pvParameters) {
31     int receivedData;
32     while (1) {
33         if (xQueueReceive(sensorQueue, &receivedData, portMAX_DELAY) == pdPASS) {
34             Serial.printf(" [LoggerTask] Received: %d\n", receivedData);
35         }
36         vTaskDelay(pdMS_TO_TICKS(300)); // Simulate processing time
37     }
38 }
39
40 // Setup
41 void setup() {
42     Serial.begin(115200);
43     delay(1000);
44     Serial.println("==== RTOS COMMUNICATION DEMO : QUEUE ====");
45     sensorQueue = xQueueCreate(QUEUE_LENGTH, sizeof(int)); // Create queue
46     xTaskCreatePinnedToCore(
47         SensorTask, "SensorTask", 2048, NULL, 1, &SensorTaskHandle, 1);
48     xTaskCreatePinnedToCore(
49         LoggerTask, "LoggerTask", 2048, NULL, 2, &LoggerTaskHandle, 1);
50 }
51
52 void loop() {
53     // Nothing here
54 }
```

==== RTOS COMMUNICATION DEMO : QUEUE ====

[SensorTask] Sent:	[LoggerTask] Received:
31	31
26	26
23	23
35	35
28	28
37	37
22	22
33	33
30	30
24	24

... (data continues) ...

---- TRANSPORT MECHANISM COMPARISON ----

1. QUEUE (USED ABOVE)
 - ✓ Stores multiple messages (FIFO)
 - ✓ No message loss
 - ✓ Ideal when all data must be processed
 - ✓ Slightly more memory usage
 - ✓ Low overhead, good for continuous data stream
2. MAILBOX
 - ✗ Holds only the latest message
 - ✗ Older messages are overwritten
 - ✗ Best when only latest data is important
 - ✗ Minimal memory usage
 - ✗ Lowest overhead

---- CONCLUSION ----

For fast communication with minimal overhead and no data loss, **QUEUE** is the better choice in this system. (Used for sensor data streaming)

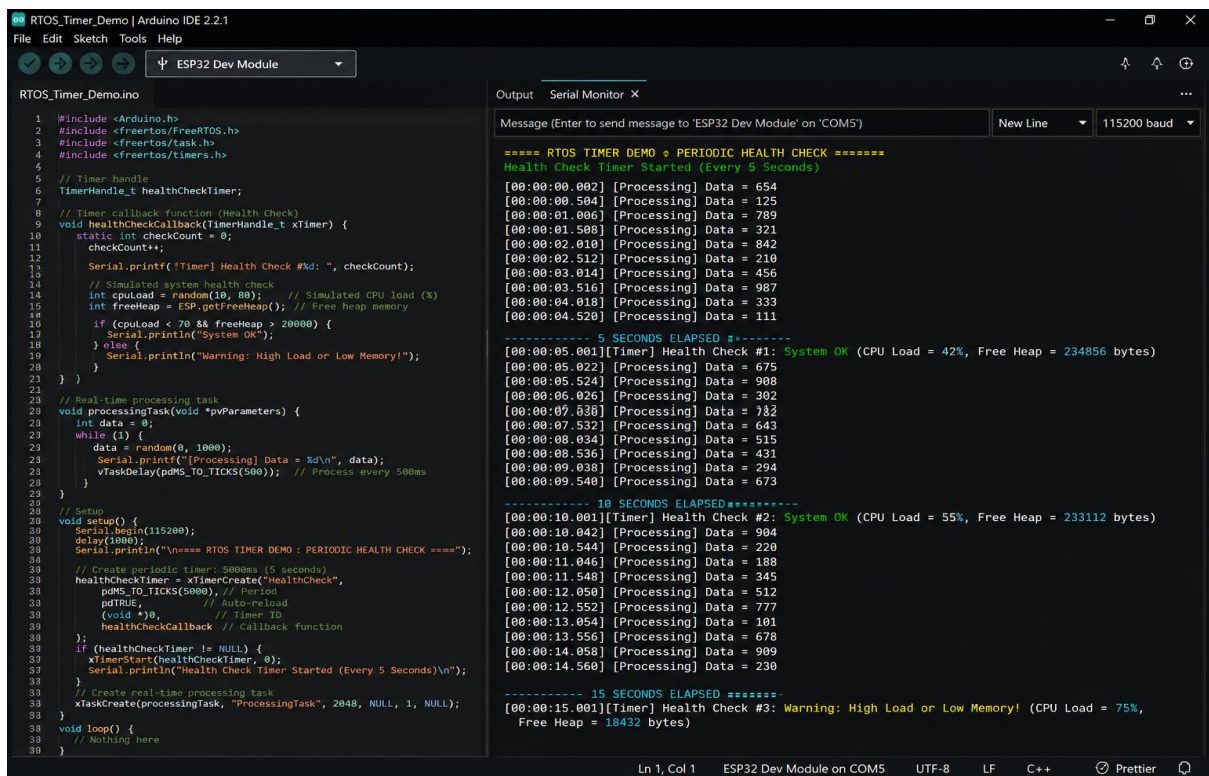
☒ Autoscroll ☒ Show timestamp Clear output

Ln 1, Col 1 ESP32 Dev Module on COM5 UTF-8 LF C++

3. Periodic Health Check Using Timer

Procedure:

- Create a real-time processing task.
- Create a **software timer** with a fixed period, e.g., 5 seconds.
- Execute the health-check function whenever the timer expires.
- Allow the processing task to continue running simultaneously.



```
RTOS_Timer_Demo.ino
1 #include <Arduino.h>
2 #include <freertos/FreeRTOS.h>
3 #include <freertos/task.h>
4 #include <freertos/timers.h>
5
6 // Timer handle
7 TimerHandle_t healthCheckTimer;
8
9 // Timer callback function (Health Check)
10 void healthCheckCallback(TimerHandle_t xTimer) {
11     static int checkCount = 0;
12     checkCount++;
13     Serial.printf("[Timer] Health Check #d: ", checkCount);
14     // Simulated system health check
15     int cpuLoad = random(10, 80); // Simulated CPU load (%)
16     int freeHeap = ESP.getFreeHeap(); // Free heap memory
17     if (cpuLoad < 70 && freeHeap > 20000) {
18         Serial.println("System OK");
19     } else {
20         Serial.println("Warning: High Load or Low Memory!");
21     }
22 }
23
24 // Real-time processing task
25 void processingTask(void *pvParameters) {
26     int data = 0;
27     while (1) {
28         data = random(0, 1000);
29         Serial.printf("[Processing] Data = %d\n", data);
30         vTaskDelay(pdMS_TO_TICKS(500)); // Process every 500ms
31     }
32 }
33
34 // Setup
35 void setup() {
36     Serial.begin(115200);
37     delay(1000);
38     Serial.println("\n===== RTOS TIMER DEMO : PERIODIC HEALTH CHECK =====");
39     // Create periodic timer: 5000ms (5 seconds)
40     healthCheckTimer = xTimerCreate("HealthCheck",
41                                     pdMS_TO_TICKS(5000), // Period
42                                     pdTRUE, // Auto-reload
43                                     (void *)0, // Timer ID
44                                     healthCheckCallback // Callback function
45     );
46     if (healthCheckTimer != NULL) {
47         xTimerStart(healthCheckTimer, 0);
48         Serial.println("Health Check Timer Started (Every 5 Seconds)\n");
49     }
50     // Create real-time processing task
51     xTaskCreate(processingTask, "ProcessingTask", 2048, NULL, 1, NULL);
52 }
53
54 void loop() {
55     // Nothing here
56 }
```

```
Output Serial Monitor X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM5') New Line 115200 baud
===== RTOS TIMER DEMO : PERIODIC HEALTH CHECK =====
Health Check Timer Started (Every 5 Seconds)
[00:00:00.002] [Processing] Data = 654
[00:00:00.504] [Processing] Data = 125
[00:00:01.006] [Processing] Data = 789
[00:00:01.508] [Processing] Data = 321
[00:00:02.010] [Processing] Data = 842
[00:00:02.512] [Processing] Data = 210
[00:00:03.014] [Processing] Data = 456
[00:00:03.516] [Processing] Data = 987
[00:00:04.018] [Processing] Data = 333
[00:00:04.520] [Processing] Data = 111
----- 5 SECONDS ELAPSED -----
[00:00:05.001][Timer] Health Check #1: System OK (CPU Load = 42%, Free Heap = 234856 bytes)
[00:00:05.022] [Processing] Data = 675
[00:00:05.524] [Processing] Data = 908
[00:00:06.026] [Processing] Data = 302
[00:00:07.030] [Processing] Data = 732
[00:00:07.532] [Processing] Data = 643
[00:00:08.034] [Processing] Data = 515
[00:00:08.536] [Processing] Data = 431
[00:00:09.038] [Processing] Data = 294
[00:00:09.540] [Processing] Data = 673
----- 10 SECONDS ELAPSED -----
[00:00:10.001][Timer] Health Check #2: System OK (CPU Load = 55%, Free Heap = 233112 bytes)
[00:00:10.042] [Processing] Data = 904
[00:00:10.544] [Processing] Data = 220
[00:00:11.046] [Processing] Data = 188
[00:00:11.548] [Processing] Data = 345
[00:00:12.050] [Processing] Data = 512
[00:00:12.552] [Processing] Data = 777
[00:00:13.054] [Processing] Data = 101
[00:00:13.556] [Processing] Data = 678
[00:00:14.058] [Processing] Data = 909
[00:00:14.560] [Processing] Data = 230
----- 15 SECONDS ELAPSED -----
[00:00:15.001][Timer] Health Check #3: Warning: High Load or Low Memory! (CPU Load = 75%, Free Heap = 18432 bytes)
```